Introduction

- electron is an open source framework that is used to build cross platform desktop applications using web technologies like html, css and js.

Pattern

01 `Architecture & security` - Main, renderer, preload, contextBridge, sandbox 02 `IPC communication` - invoke/handle, send/on, validation, errors 03 `Lifecycle & debugging` - Windows, crashes, memory leaks, performance 04 `Build & packaging` - Vite, installers, cross-platform deployment

why desktop app ?

- writing conditional code because of browser differences.
- access to file system is also limited
- slow network connectivity

why web tech ?

- huge support community
- cross platform

how does electron work ?

- chromium + node js ---> electron

basic code

```
const {app, BrowserWindow} = require('electron')
const path = require('path')
const url = require('url')


let win;

function createWindow () {

  win = new BrowserWindow({width: 800, height: 600})

  win.loadURL(url.format({
    pathname: path.join(__dirname, 'index.html'),
    protocol: 'file:',
    slashes: true
  }));

   win.webContents.on(
    "render-process-gone",
    (event, details) => {
      console.error("Renderer process terminated:", {
        reason: details.reason,
        exitCode: details.exitCode,
      });

      // Decide how to recover based on the reason.
    }
  );


  win.on('closed', () => {
    win = null
  });
}

app.on('ready', createWindow);
```

```
app.on('window-all-closed', () => {
  if (process.platform !== 'darwin') {
    app.quit();
  }
});
```

Main and Renderer process

- 1 main process and 1 or more renderer process
- Main process
    - it is the entry point of electron app
    - it runs with node js and has access to electron OS APIs.
    - it creates and manages app window using BrowserWindow.
    - it is kind of backend/controller of our desktop app.
- Renderer process
    - every browserwindow has its own renderer process.
    - it is responsible for dispalying the UI - Html, css, js/react etc.
    - it is kind of frontend/ui of our desktop app.
    - by default it doesn't have direct access to node.js or privileged elctron apis.
- Communication: electron app -> main process `controls the app and OS` -> Preload + IPC `bridge between them` -> renderer process `displays and handles the UI`
- `app.whenReady()` resolves when Electron has initialized and is ready to create browser windows.
- The `window-all-closed` event fires when all windows are closed; on Windows and Linux, applications commonly quit at this point.
- On macOS, applications commonly remain active, and the `activate` event lets us recreate a window when the user reopens the app from the Dock.

BrowserWindow

- BrowserWindow is the object used by Electron's Main Process to create and control application windows.
- Each BrowserWindow contains a Renderer Process that displays the web UI.
- The Main Process manages the BrowserWindow and therefore manages the renderer associated with it.
- win = new BrowserWindow({ width: 400, height: 600, backgroundColor: '#eeee', frame: false, title: 'Parent' });
- for child window we can also apply property: `modal:true` -> for completing child first and then can access parent.

IPC [Inter process communication]

- Preload is a controlled bridge between Renderer and Main.
- The preload script can use IPC APIs to send requests to the main process, where privileged file operations can be performed.
- contextBridge exposes selected APIs to the renderer.
- I explicitly configure `contextIsolation`: true to separate the preload context from the renderer's JavaScript context, `nodeIntegration`: false to prevent renderer code from directly accessing Node.js APIs, and `sandbox`: true to restrict the renderer's operating-system access. I expose only narrowly scoped APIs through the preload script and validate IPC requests in the main process.

```
// preload.js
const { contextBridge, ipcRenderer } = require('electron'
```

```
contextBridge.exposeInMainWorld('electronAPI', {
  getAppVersion: () => ipcRenderer.invoke('get-app-versio
})

// main.js
ipcMain.handle('get-app-version', () => {
  return app.getVersion()
})
```

now renderer can do: `const version = await window.electronAPI.getAppVersion()`

without having direct access to ipcRenderer or Node.js.

```
User clicks button
       ↓
Renderer
       ↓
window.electronAPI.showError()
       ↓
Preload
       ↓
ipcRenderer
       ↓
Main Process
       ↓
Native Electron/OS API
```

How do you create an application menu in Electron?

Menus are managed from the Main Process using Electron's Menu API. We can define a menu template with items and submenus, create it with Menu.buildFromTemplate(), and install it using Menu setApplicationMenu(). For standard operations like copy, paste, undo, redo, and DevTools, I would use Electron's built-in roles rather than implementing those behaviors manually.

Menu template ↓ Menu.buildFromTemplate() ↓

Menu.setApplicationMenu()

```
const menu = Menu.buildFromTemplate(template)
Menu.setApplicationMenu(menu)
```

Menus

What types of menus does Electron support?

- Application Menu = top-level application menu like File, Edit, View, Help.
- Context Menu = right-click menu
- Tray Menu
- macOS Dock Menu

Menu items can have

- label = text displayed to the user
- submenu = nested menu items
- click = handler when the item is clicked
- type: "separator" = visual separator between items
- role = Electron's built-in behavior

Example of context menu

```
const contextMenu = new Menu()

contextMenu.append(
  new MenuItem({
    label: 'Hello',
    click() {
      console.log('Context menu item clicked')
    }
```

```
  })
)

win.webContents.on('context-menu', (event, params) => {
  contextMenu.popup({
    x: params.x,
    y: params.y
  })
})
```

- Context menus can also use Electron's predefined roles
- params.x and params.y give the mouse cursor's coordinates where the right-click. They are used to show the context menu exactly where the user right-clicked.

Accelerators and global shortcuts

Accelerator = keyboard shortcut for your Electron app.

```
 {
    label: 'Hello',
    click() {
      console.log('Context menu item clicked')
    },
    accelerator: 'CmdOrCtrl + Shift + H'
 }
```

Global Shortcut = keyboard shortcut that works even when the app is not focused.

```
globalShortcut.register('Alt+1', () => {
  win.show()
})

app.on('will-quit', () => {
  globalShortcut.unregisterAll()
})
```

Shell

shell lets Electron interact with the user's native OS for files, folders, and external URLs.

```
shell.showItemInFolder('/path/demo.txt')

shell.openPath('/path/image.jpg')

shell.openExternal('https://electronjs.org')
```

Tray module

Tray icon = keeps your Electron app accessible from the system tray/ menu bar.

```
const tray = new Tray(iconPath)

const contextMenu = Menu.buildFromTemplate(template)
tray.setContextMenu(contextMenu) //attach actions/options

tray.setToolTip('My App')
```

CRUD

Electron can perform Create, Read, Update, Delete (CRUD) operations on files using Node.js fs and path.

Operation	API	Purpose
Create / **Update**	`fs.writeFile()`	Creates a file or overwrites existing content
Read	`fs.readFile()`	Reads file contents

Delete	`fs.unlink()`	Deletes a file

Memory leaks and performance

First, I would reproduce the slowdown and monitor CPU and memory usage over time. I'd use Chrome DevTools Performance and Memory panels to inspect renderer performance, heap snapshots, and retained objects, and check React Profiler for unnecessary renders. I'd also inspect main-process memory, IPC traffic, event listeners, timers, and WebSocket cleanup. Finally, I'd fix the identified cause and verify that memory usage stabilizes during a long-running test.

Renderer crash

I would listen for the render-process-gone event on the webContents object to detect when the renderer process terminates unexpectedly. I'd log the termination details, show a recovery or error screen if possible, and recreate or reload the renderer when appropriate. Before retrying, I'd investigate the crash reason and prevent an infinite restart loop. I'd also make sure unsaved user data is protected where possible.

Electron packaging and deployment evaluation

Packaging an Electron app

I would first build the React frontend using the project's build script, then use Electron Builder or Electron Forge to package the Electron application. I'd configure the application entry point, frontend output files, icons, target operating systems, and installer formats. Finally, I'd test the packaged application on each target platform.

Development vs. production blank screen

First, I'd inspect application logs and renderer console errors. I'd verify that the production frontend build succeeded, that the packaged files are included, and that Electron loads the correct file or URL. I'd also check Vite's base path, asset paths, preload paths, and any runtime errors that occur only in production. Then I'd reproduce the issue in the packaged application and test the fix.

Cross-platform compatibility

I'd reproduce the problem on the affected operating system and inspect its logs. I'd check file paths and case sensitivity, file permissions, OS-specific APIs, native module compatibility, architecture differences, and installer configuration. I'd test a packaged build on each supported operating system and include platform-specific checks in the release pipeline